



Operating Systems & Networks CTEC1004C

Lecture - 3 : Finite state automata & Introduction to data encryption

Dr. Sameer Jha, SFHEA

Faculty of Computing

De Montfort University Cambodia

Index

- Finite State Machines
- Coffee (Finite State) Machine
- Encryption
- Key Objective of Encryption Data
- Why to Encrypt?
- How Encryption Works
- Cryptography
- Digital Signatures
- Message Digests

Finite State Machines

“What’s in a name?”

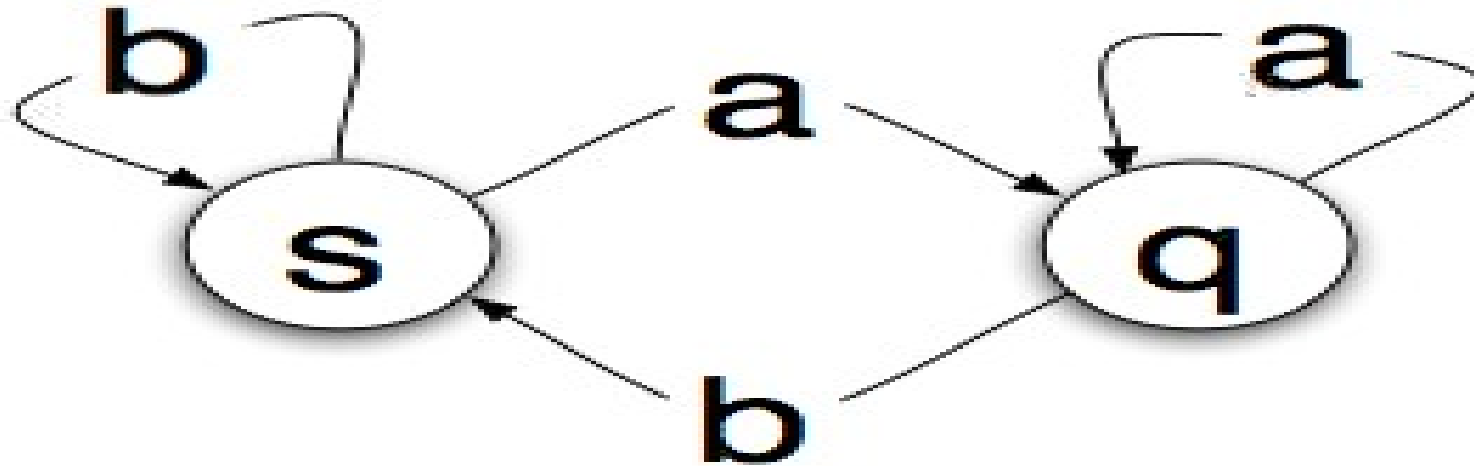


- Er, for one, the answer to what Finite State Machines are.....
 - Finite State Machine is an abstract machine that can be in exactly one of a finite number of “states” at any given time, to put it plainly.
 - State - A *state* is a description of the status of a system

Finite State Machines contd.

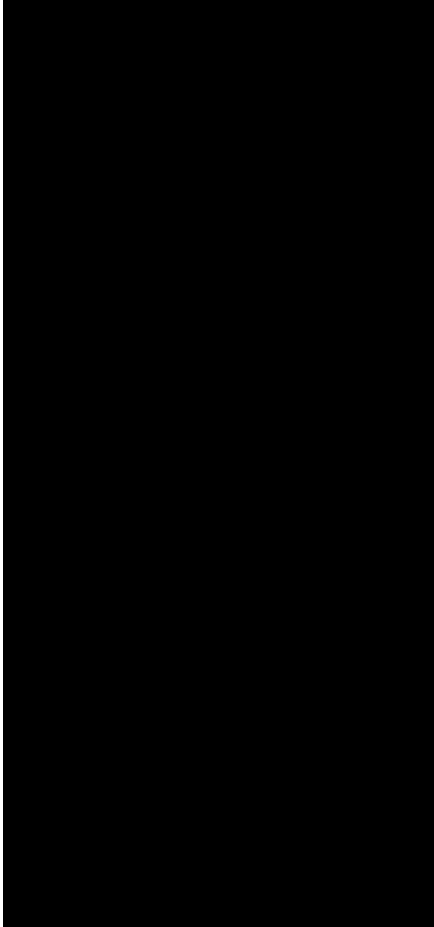
- A Finite State Machine does a ***transition*** when a condition is fulfilled or when an event is received.
- A transition is a set of actions that the machine does.
In some FSMs, it is also possible to associate actions with a state:
- an entry action: performed when entering the state, and
- an exit action: performed when exiting the state.

Finite State Machine Representation



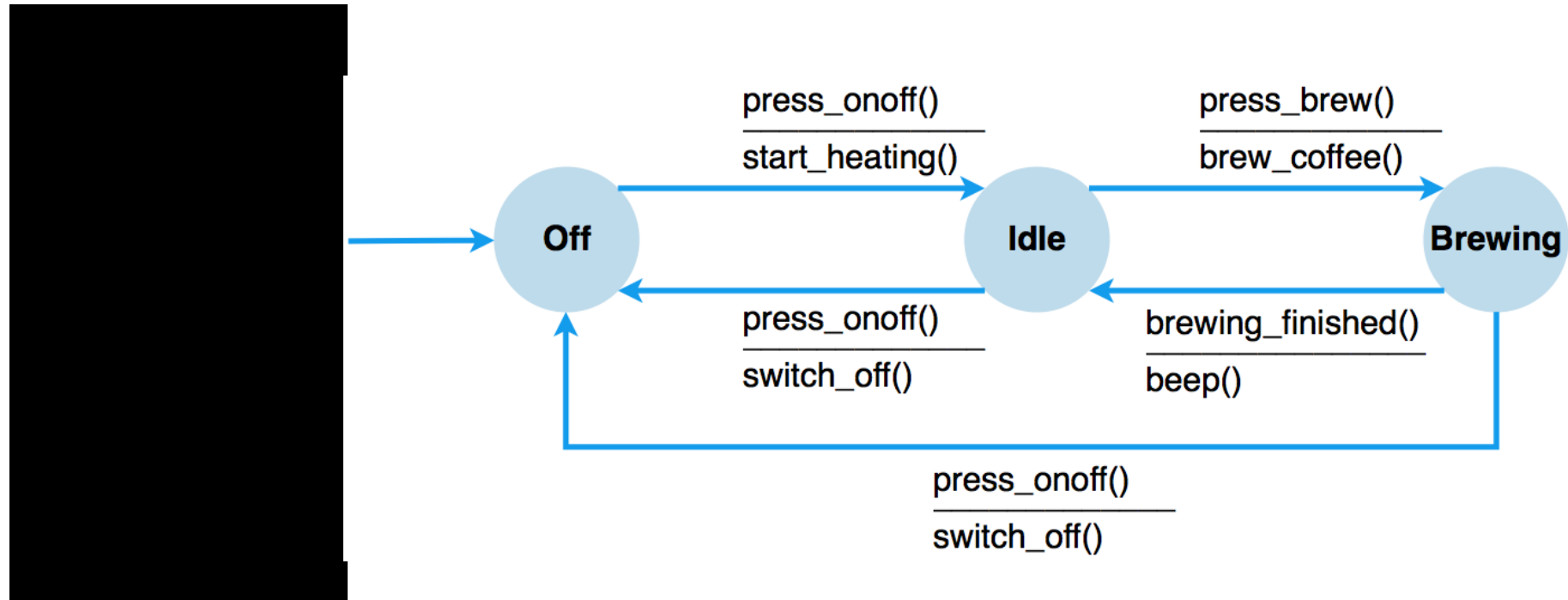
The circles are “**states**” that the machine can be in. The arrows are the **transitions**. So, if you are in state s and read an ‘a’, you’ll transition to state q. If you read a ‘b’, you’ll stay in state s

Coffee (Finite State) Machine



- Two buttons
- What states can the machine be in?
- What events cause changes between states?
- What actions should be taken on state transition?

Coffee (Finite State) Machine



Encryption

Data Encryption

- Data encryption is the process of converting readable information (plaintext) into an unreadable format (ciphertext) to protect it from unauthorized access.
- It is a method of preserving data confidentiality by transforming it into ciphertext, which can only be decoded using a unique decryption key produced at the time of the encryption or before it. The conversion of plaintext into ciphertext is known as encryption. By using encryption keys and mathematical algorithms, the data is scrambled so that anyone intercepting it without the proper key cannot understand the contents.

Data Encryption (Cont.)

- When the intended recipient receives the encrypted data, they use the matching decryption key to return it to its original, readable form. This approach ensures that sensitive information such as personal details, financial data, or confidential communications remains secure as it travels over networks or is stored on devices.

Key Objective of Encryption Data

-

Why Encrypt?

....because there's bad guys around.

If information is un-encrypted, they can –

1. Eavesdrop

→ intercept messages

→ actively insert messages into connection

2. Impersonate

→ can fake (spoof) source address in packet (or any field in packet)

Why Encrypt?

3. Hijack

- “take over” ongoing connection by removing sender or receiver,
- inserting himself in place

4. Denial of Service (DDoS)

- prevent service from being used by others by overloading resources

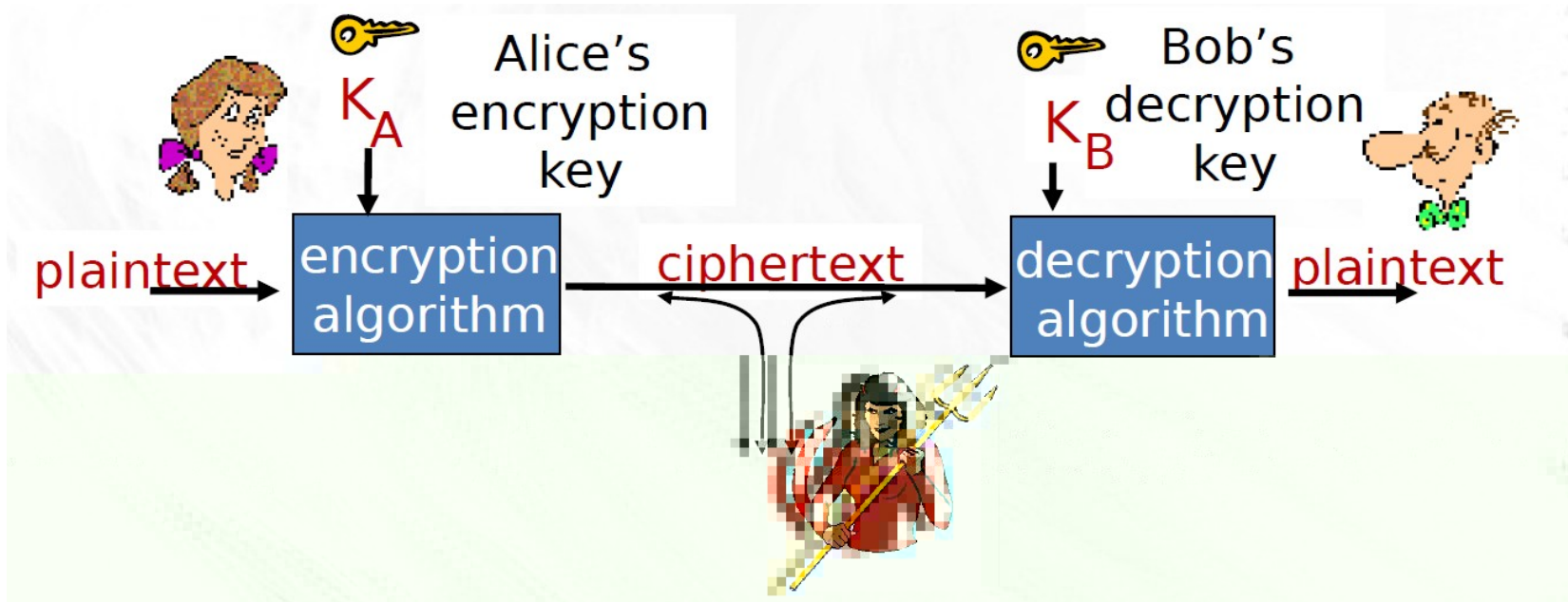
How Encryption Works

- Original information, or plain text, might be something as simple as "Hello, world!" As cipher text, this might appear as something confusing like 7*#0+gvU2x—something seemingly random or unrelated to the original plaintext.
- Encryption, however, is a logical process, whereby the party receiving the encrypted data , but also in possession of the key-can simply decrypt the data and turn it back into plaintext.

How Encryption Works (Cont.)

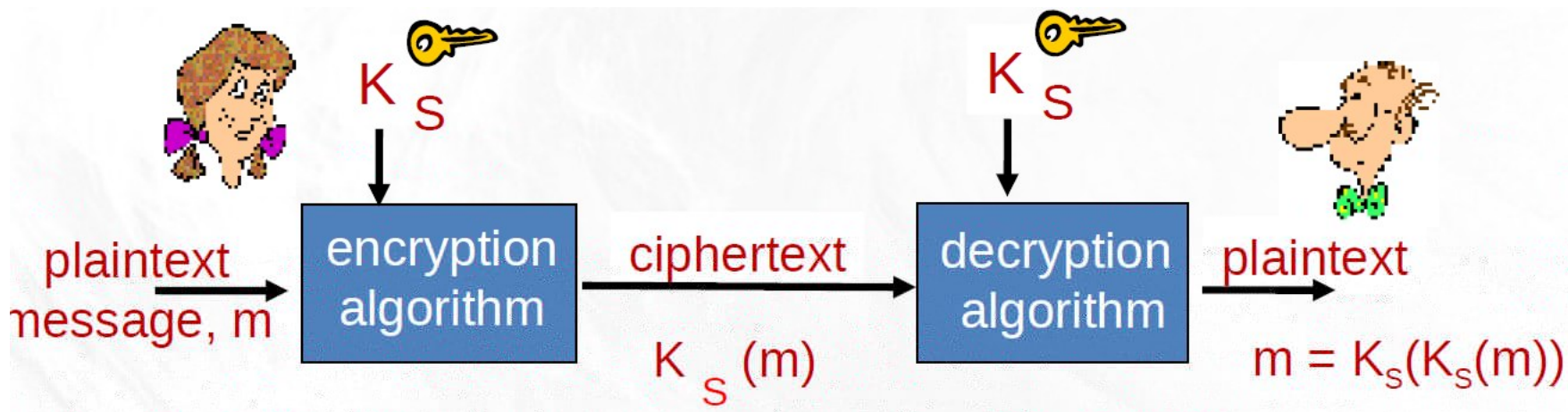
- An encryption algorithm is a mathematical formula used to transform plaintext (data) into ciphertext. An algorithm will use the key to alter the data in a predictable way. Even though the encrypted data appears to be random, it can actually be turned back into plaintext by using the key again. Some commonly used encryption algorithms include Blowfish, Advanced Encryption Standard (AES), Rivest Cipher 4 (RC4), RC5, RC6, Data Encryption Standard (DES), and Twofish.

Cryptography



- m - plaintext message
- $K_A(m)$ ciphertext, encrypted with key K_A
- $m = K_B(K_A(m))$

Symmetric Key Cryptography



- symmetric key crypto:
 - Bob & Alice share same (symmetric) key: K_S
 - key is knowing substitution pattern in mono alphabetic substitution cipher
- Q: how do Bob and Alice agree on key value?

Simple encryption scheme

substitution cipher: substituting one thing for another

monoalphabetic cipher: substitute one letter for another

plaintext: a b c d e f g h i j k l m n o p q r s t u v w x y z

ciphertext: m n b v c x z a s d f g h j k l p o i u y t r e w q

e.g.: Plaintext: bob. i love you. alice
 ciphertext: nkn. s gktc wky. mgsbc



Encryption key: mapping from set of 26 letters to set of 26 letters



AES: Advanced Encryption Standard

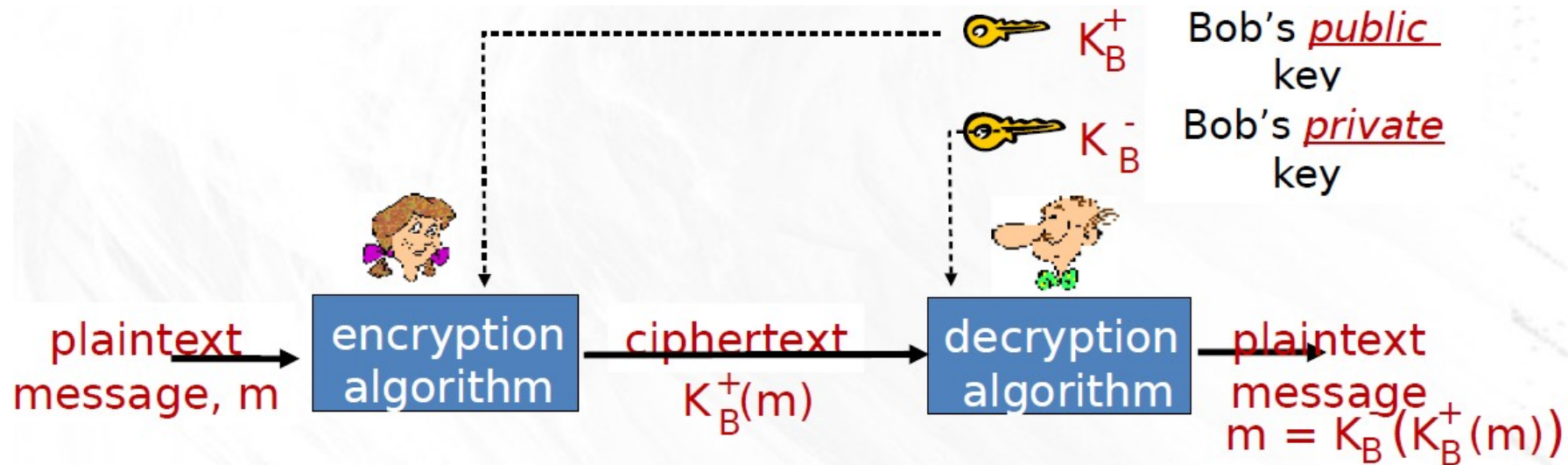
- Symmetric-key NIST standard
 - replaced DES (Nov 2001)
- Processes data in 128 bit blocks
- 128, 192, or 256 bit keys
- Brute force decryption (try each key) taking 1 sec on DES, takes 149 trillion years for AES
- Very fast, can be implemented in hardware



AES: Advanced Encryption Standard

- Symmetric-key NIST standard
 - replaced DES (Nov 2001)
- Processes data in 128 bit blocks
- 128, 192, or 256 bit keys
- Brute force decryption (try each key) taking 1 sec on DES, takes 149 trillion years for AES
- Very fast, can be implemented in hardware

Public Key Cryptography



Radically different approach [Diffie-Hellman76, RSA78]

- Sender & receiver do not share secret key
- Public encryption key known to all
- Private decryption key known only to receiver

Public Key Encryption Algorithms

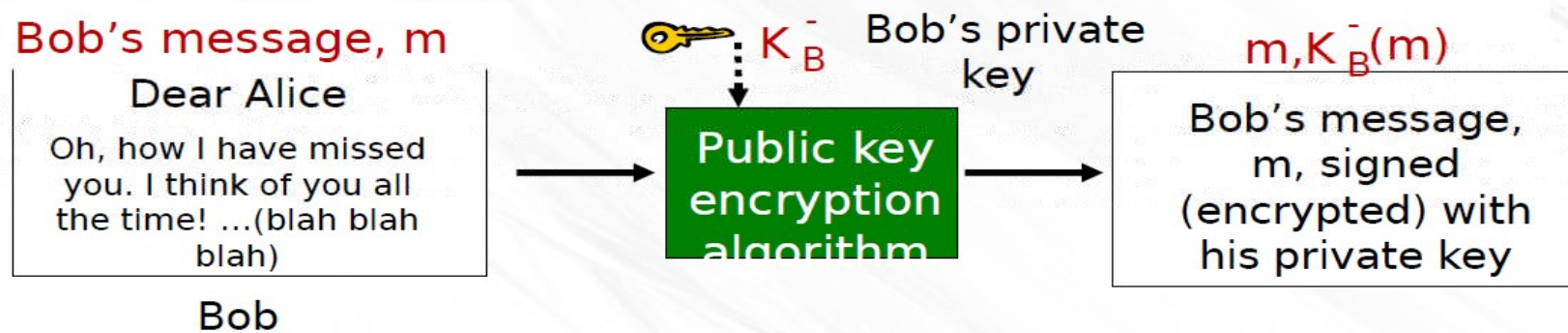
- Requirements:
- 1. need $KB+(m)$ and $KB-(m)$ such that
 - $KB-(KB+(m)) = m$
- 2. given public key $KB+$, it should be
- impossible to compute private key KB Public

Public Key Encryption Algorithms

- RSA
 - Rivest, Shamir, Adelson algorithm
- Encryption using RSA is computationally expensive
- Strategy
 - use public key crypto to establish secure connection
 - then use symmetric session key to encrypt data

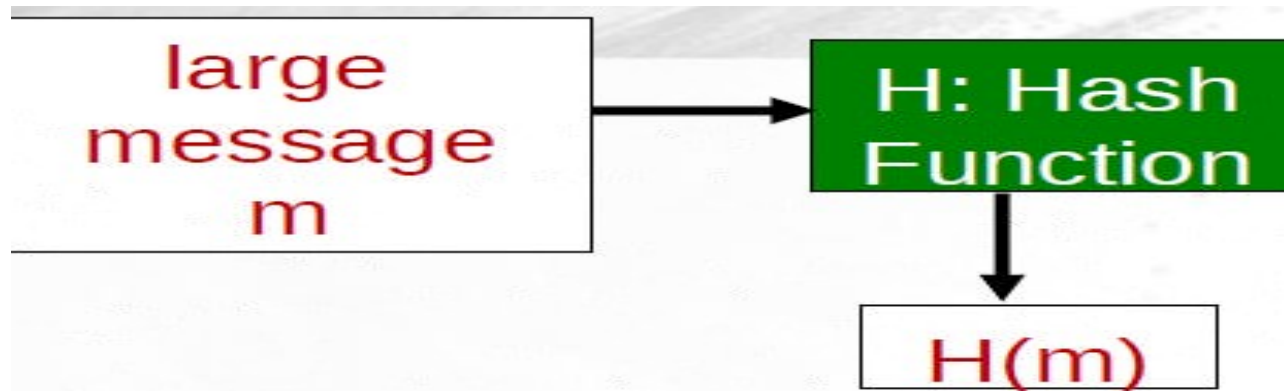
Digital Signatures

- simple digital signature for message m :
- Bob signs m by encrypting with his private key K_B^- , creating “signed” message, $K_B^-(m)$
- Alice can take m , and signature $K_B^-(m)$ to Court and prove



Message Digests

- Computationally expensive to public-key-encrypt long messages
- Goal
 - fixed-length, easy- to-compute digital “fingerprint”
- Apply hash function H to m
 - get fixed size message digest, $H(m)$



Message Digests

Hash function properties:

- many-to-1
- produces fixed-size message digest (fingerprint)
- given message digest x , computationally infeasible to find m such that $x = H(m)$

Internet checksum: poor crypto hash function

- Internet checksum has some properties of hash function:
- Produces fixed length digest (16-bit sum) of message
- Is many-to-one
- But given message with given hash value, it is easy to find another message with same hash value:

<u>message</u>	<u>ASCII format</u>
I O U 1	49 4F 55 31
0 0 . 9	30 30 2E 39

<u>message</u>	<u>ASCII format</u>
I O U 9	49 4F 55 39
0 0 1	30 30 2F 31

9 3 0 3	39 33 30 33
32 01 02 AC	32 01 02 AC

9 3 0 3	39 33 30 33
32 01 02 AC	32 01 02 AC

different messages
but identical checksums!

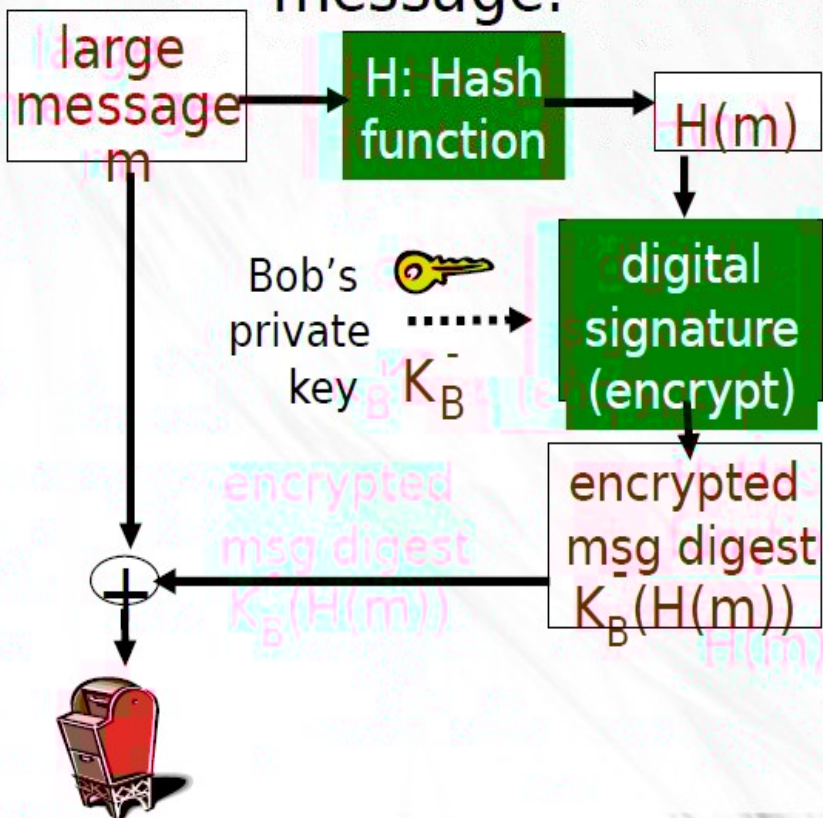
Hash function algorithms

- MD5 hash function used to be widely used (RFC 1321)
 - computes 128-bit message digest in 4-step process.
 - BUT: has been shown to be easily compromised!
- Nowadays: use SHA family
 - SHA-256 and SHA-512 give 256 and 512 bit message digests, respectively
- SHA3
 - NIST 2015
- <https://issha3brokenyet.com/>

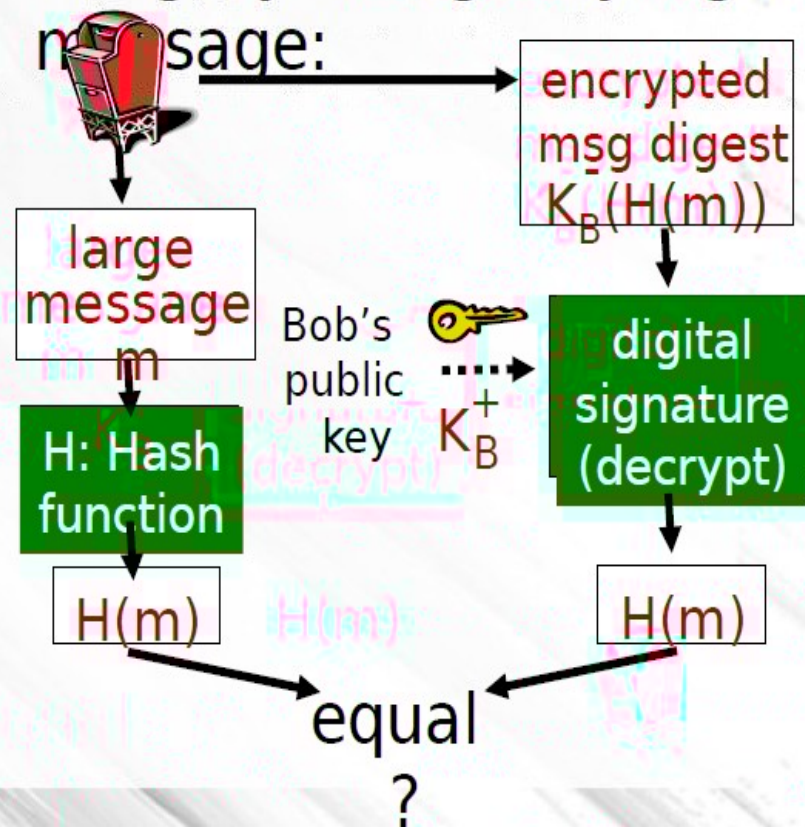


Digital signature = signed message digest

Bob sends digitally signed message:



Alice verifies signature, integrity of digitally signed message:





THANK YOU!